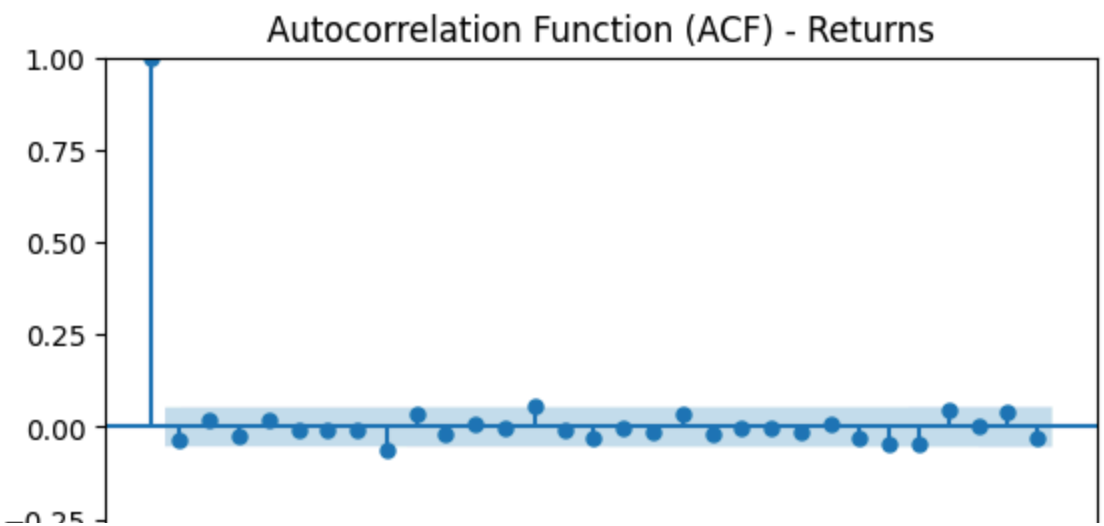
Step 1a

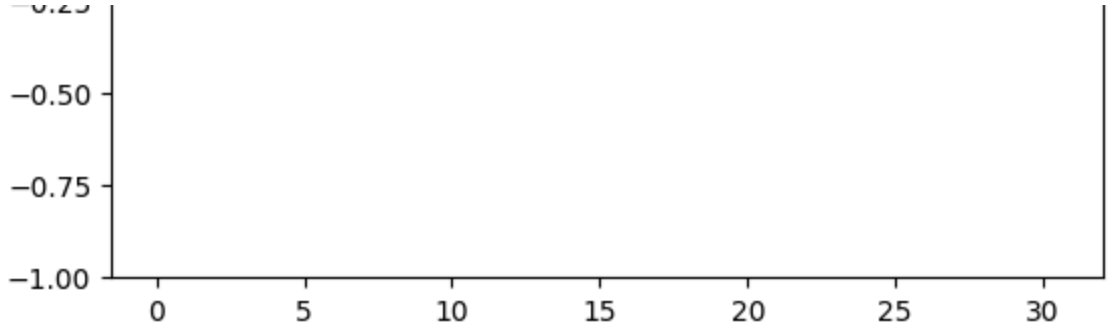
```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.stats.stattools import jarque_bera
from scipy.stats import skew, kurtosis

# -----------------------
# Parameters
# -----------------------
TICKER_SYMBOL = "AMZN"
START_DATE = "2020-01-01"
END_DATE = "2025-06-30"

# -----------------------
# 1. Download price data
# -----------------------
data = yf.download(TICKER_SYMBOL, start=START_DATE, end=END_DATE)

# Ensure Close is a Series, not a DataFrame
prices = data["Close"].squeeze()   # <--- FIX HERE

# -----------------------
# 2. Compute log returns
# -----------------------
returns = np.log(prices / prices.shift(1)).dropna()

print("\n====================")
print(" BASIC INFORMATION ")
print("====================")
print(f"Ticker: {TICKER_SYMBOL}")
print(f"Periods: {len(returns)} observations\n")
```

```
# -----------------------
# 3. Summary statistics
# -----------------------
mean_ret = float(returns.mean())
std_ret = float(returns.std())
skew_ret = float(skew(returns))
kurt_ret = float(kurtosis(returns))  # excess kurtosis

jb_stat, jb_p, _, _ = jarque_bera(returns)

print("====================")
print(" DISTRIBUTION STATS ")
print("====================")
print(f"Mean (daily log return): {mean_ret:.6f}")
print(f"Std (volatility):       {std_ret:.6f}")
print(f"Skewness:               {skew_ret:.4f}")
print(f"Excess Kurtosis:        {kurt_ret:.4f}")
print(f"Jarque-Bera p-value:    {jb_p:.4f}\n")

# -----------------------
# 4. ADF unit root test
# -----------------------
adf_stat, adf_p, _, _, _, _ = adfuller(returns)

print("====================")
print(" ADF STATIONARITY TEST ")
print("====================")
print(f"ADF Statistic: {adf_stat:.4f}")
print(f"p-value:       {adf_p:.4f}")
print("Reject Null (stationary) if p < 0.05\n")

# -----------------------
# 5. Autocorrelation
# -----------------------
plt.figure(figsize=(8,4))
plot_acf(returns, lags=30)
plt.title("Autocorrelation Function (ACF) - Returns")
plt.show()
```

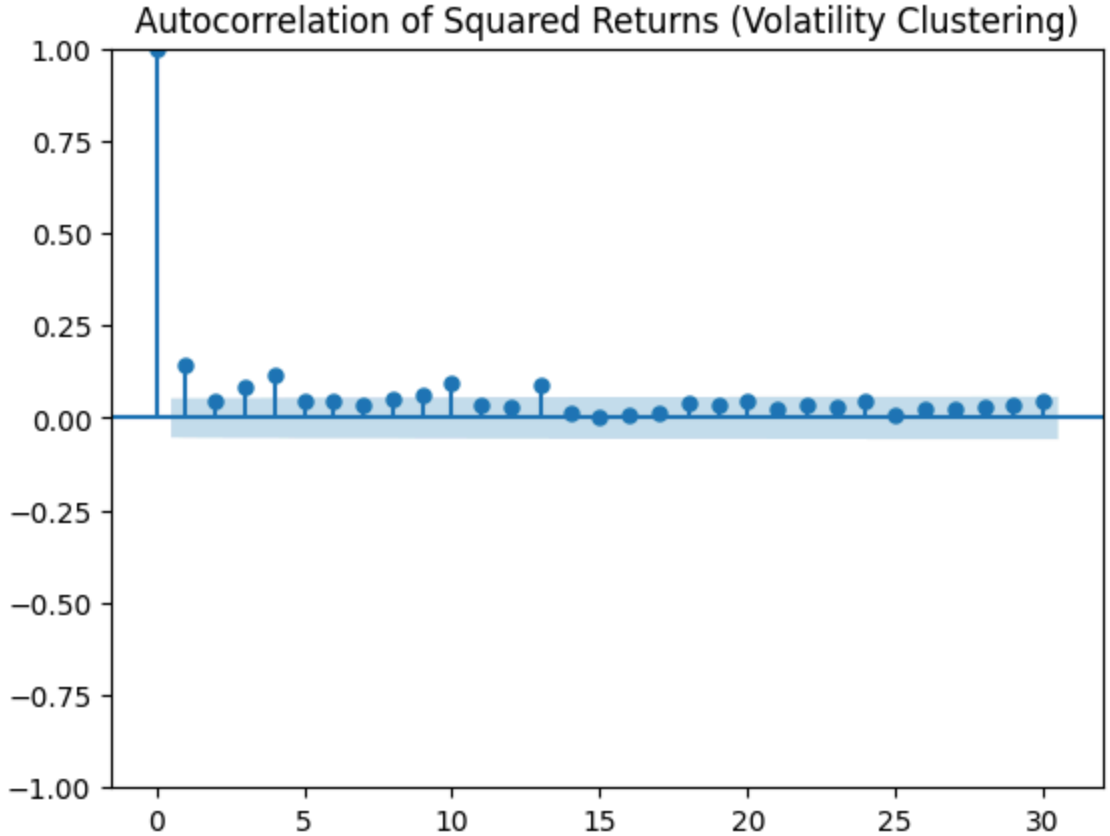
```
plt.figure(figsize=(8,4))
plot_acf(returns**2, lags=30)
plt.title("Autocorrelation of Squared Returns (Volatility Clustering)")
plt.show()

# -----------------------
# 6. Five-number summary
# -----------------------
print("====================")
print(" FIVE-NUMBER SUMMARY ")
print("====================")
print(returns.describe(percentiles=[0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99]))
```

```
/tmp/ipython-input-1121635320.py:20: FutureWarning: YF.download() has changed argument auto_adjust default to T
  data = yf.download(TICKER_SYMBOL, start=START_DATE, end=END_DATE)
[*********************100%***********************]  1 of 1 completed

====================
 BASIC INFORMATION
====================
Ticker: AMZN
Periods: 1378 observations

====================
 DISTRIBUTION STATS
====================
Mean (daily log return): 0.000621
Std (volatility):       0.022766
Skewness:               -0.0730
Excess Kurtosis:        4.1775
Jarque-Bera p-value:    0.0000

====================
 ADF STATIONARITY TEST
====================
ADF Statistic: -38.4712
p-value:       0.0000
Reject Null (stationary) if p < 0.05

<Figure size 800x400 with 0 Axes>
```






```
<Figure size 800x400 with 0 Axes>
```




```
====================
 FIVE-NUMBER SUMMARY
====================
count    1378.000000
mean        0.000621
std         0.022766
min        -0.151398
```

Step 16)

```
1%        -0.057652
5%        -0.034088
25%       -0.011290
max        0.126949
Name: AMZN, dtype: float64
```

```
# Run in terminal or notebook cell with `!` prefix
!pip install yfinance statsmodels numpy pandas matplotlib seaborn scikit-learn tensorflow pyts fracdiff
```

```
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (0.2.66)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.12/dist-packages (0.14.5)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Collecting pyts
  Downloading pyts-0.13.0-py3-none-any.whl.metadata (10 kB)
ERROR: Ignored the following versions that require a different python version: 0.9.0 Requires-Python >=3.7.12,<
ERROR: Could not find a version that satisfies the requirement fracdiff (from versions: none)
ERROR: No matching distribution found for fracdiff
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# PARAMETERS
TICKER = "BTC-USD"
MAX_OBS = 2000

# fetch
df = yf.download(TICKER, period="max", progress=False)  # or specify start/end dates
df = df[["Close"]].dropna().rename(columns={"Close":"price"})
df = df.tail(MAX_OBS).copy()
df.index = pd.to_datetime(df.index)
df.head()
```

```
/tmp/ipython-input-1984510139.py:11: FutureWarning: YF.download() has changed argument auto_adjust default to T
  df = yf.download(TICKER, period="max", progress=False)  # or specify start/end dates
```

Price	price
Ticker	BTC-USD
Date	
2020-06-05	9665.533203
2020-06-06	9653.679688
2020-06-07	9758.852539
2020-06-08	9771.489258
2020-06-09	9795.700195

Next steps: Generate code with df New interactive sheet

```
from statsmodels.tsa.stattools import adfuller

# log price and first difference (returns)
df["log_price"] = np.log(df["price"])
df["r1"] = df["log_price"].diff().dropna()

# drop initial NaN
series = df["r1"].dropna()

# ADF test
adf_res = adfuller(series, autolag="AIC")
print("ADF statistic:", adf_res[0])
print("p-value:", adf_res[1])
for k,v in adf_res[4].items():
    print("Critical value ({}): {:.4f}".format(k, v))

# Quick summary stats and plots
print(series.describe())
```

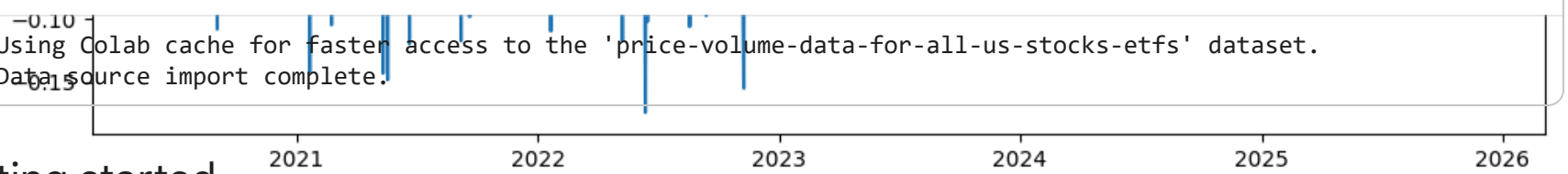
```
plt.figure(figsize=(12,4))
plt.plot(series.index, series, label="log-return (r1)")
plt.title("BTC-USD log-returns (first diff)")
plt.legend()
plt.show()

plt.figure(figsize=(6,4))
plt.hist(series, bins=60)
plt.title("Histogram of log-returns")
plt.show()
```

```
ADF statistic: -46.07273539451245
p-value: 0.0
Critical value (1%): -3.4336
Critical value (5%): -2.8630
Critical value (10%): -2.5675
count    1999.000000
mean        0.001105
std         0.030401
min        -0.174053
25%        -0.012836
50%         0.000449
75%         0.015007
max         0.171821
Name: r1, dtype: float64
```

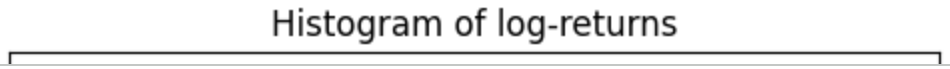
Step 1(c)

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.
import kagglehub
borismarjanovic_price_volume_data_for_all_us_stocks_etfs_path = kagglehub.dataset_download('borismarjanovic/pr

print('Data source import complete.')
```

```
Using Colab cache for faster access to the 'price-volume-data-for-all-us-stocks-etfs' dataset.
Data source import complete.
```

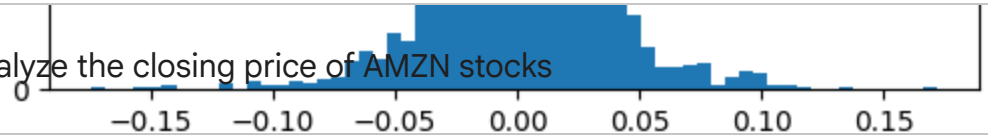


Getting started




```
import numpy as np
import pandas as pd
import gc
import os
import random
```

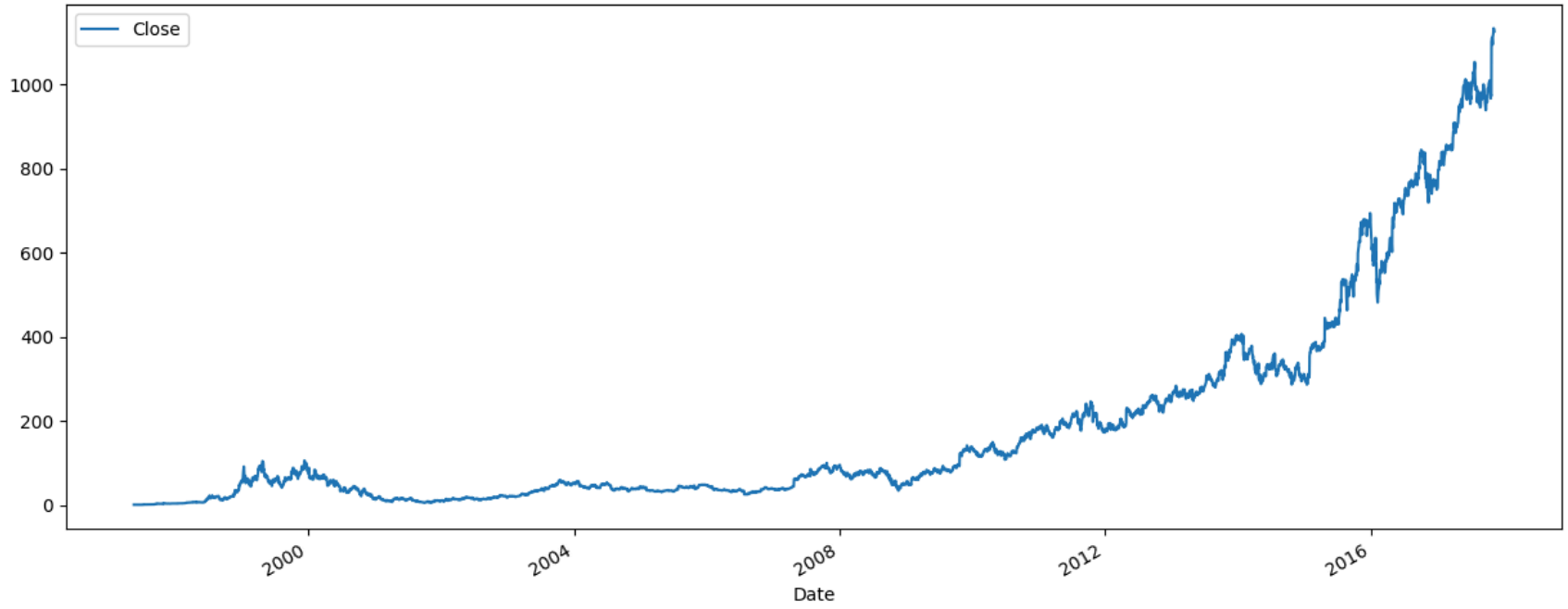
```
import copy
import matplotlib.pyplot as plt
import pandas
import statsmodels
from datetime import datetime
from statsmodels.tsa.stattools import adfuller
```



We'll analyze the closing price of AMZN stocks

```
dateparse = lambda d: datetime.strptime(d, '%Y-%m-%d')
df=pd.read_csv('/kaggle/input/price-volume-data-for-all-us-stocks-etfs/Data/Stocks/amzn.us.txt',parse_dates=[
df=df.drop(columns=['Open','High','Low','Volume','OpenInt'])
df.plot(figsize=(15,6))
plt.show()
```

```
/tmp/ipython-input-1763827541.py:2: FutureWarning: The argument 'date_parser' is deprecated and will be removed
  df=pd.read_csv('/kaggle/input/price-volume-data-for-all-us-stocks-etfs/Data/Stocks/amzn.us.txt',parse_dates=[
```



Usually univariate time series analysis is done on a stationary series, so first off we'll check for stationarity using the ADF test

```
adftest=adfuller(df['Close'])
print('ADF Statistic: %f' % adftest[0])
print('ADF p-value: %f' % adftest[1])
```

```
ADF Statistic: 4.573103
ADF p-value: 1.000000
```

We cannot reject the null hypothesis of non-stationarity. A popular way to try and circumvent this is to use the logarithm of the original series, so lets see how that works out.

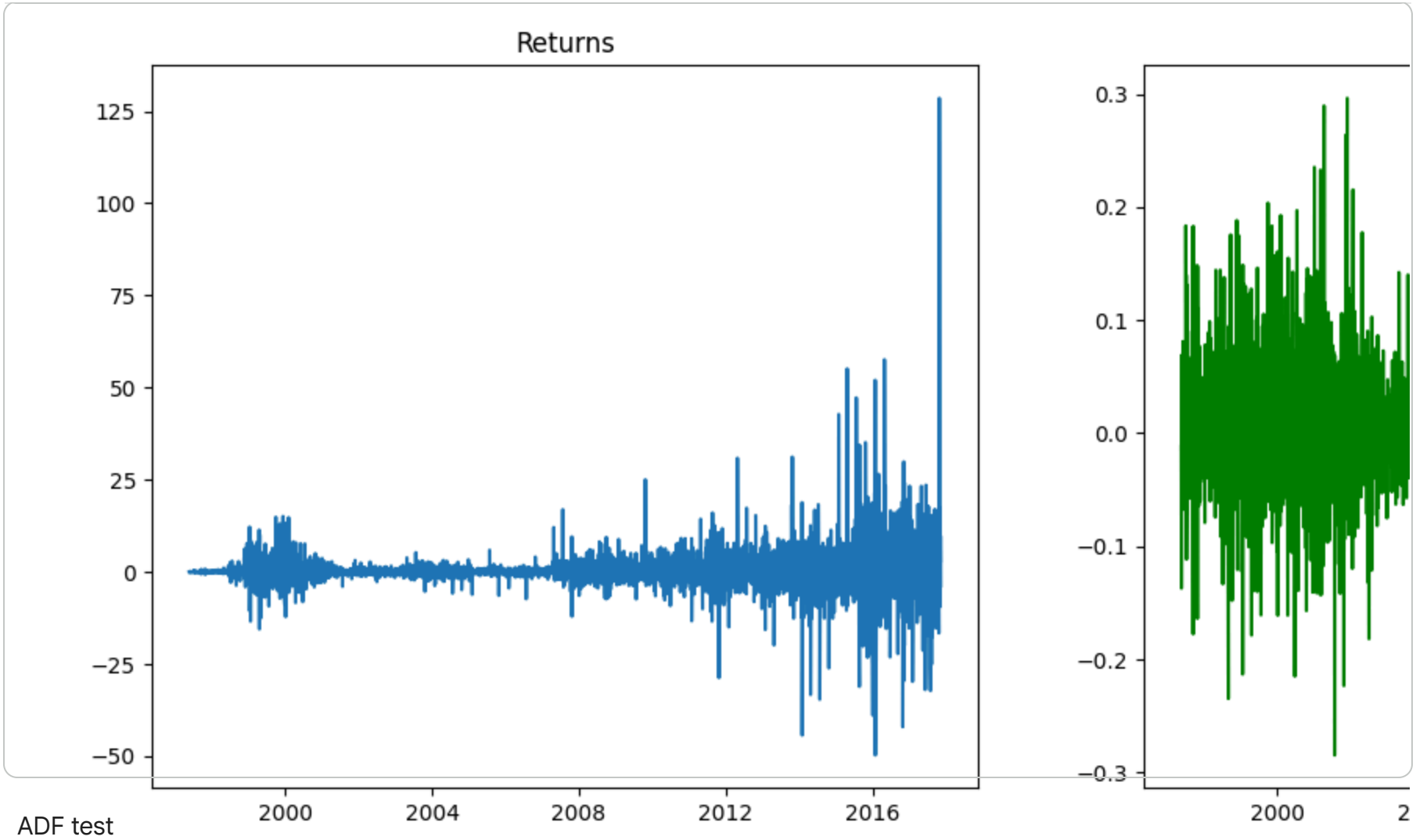
```
df_log=pd.DataFrame(np.log(df['Close']),index=df.index)
plt.figure(figsize=(15,6))
plt.plot(df_log)
plt.title('Log Close')
plt.show()
```

```
adftest=adfuller(df_log['Close'])
print('ADF Statistic: %f' % adftest[0])
print('ADF p-value: %f' % adftest[1])
```

```
ADF Statistic: -1.946347
ADF p-value: 0.310582
```

Traditional Differencing

```
df['Returns']=df['Close']-df['Close'].shift(1)
df_log['Log_Returns']=df_log['Close']-df_log['Close'].shift(1)
plt.figure(figsize=(15,6))
plt.subplot(1,2,1)
plt.plot(df['Returns'])
plt.title('Returns')
plt.subplot(1,2,2)
plt.plot(df_log['Log_Returns'],'g-')
plt.title('Log Returns')
plt.show()
```



ADF test

```
adftest_ret=adfuller(df['Returns'].dropna())
print('Returns ADF Statistic: %f' % adftest_ret[0])
print('Returns ADF p-value: %f' % adftest_ret[1])
adftest_lret=adfuller(df_log['Log_Returns'].dropna())
print('Log-Returns ADF Statistic: %f' % adftest_lret[0])
print('Log-Returns ADF p-value: %f' % adftest_lret[1])
```

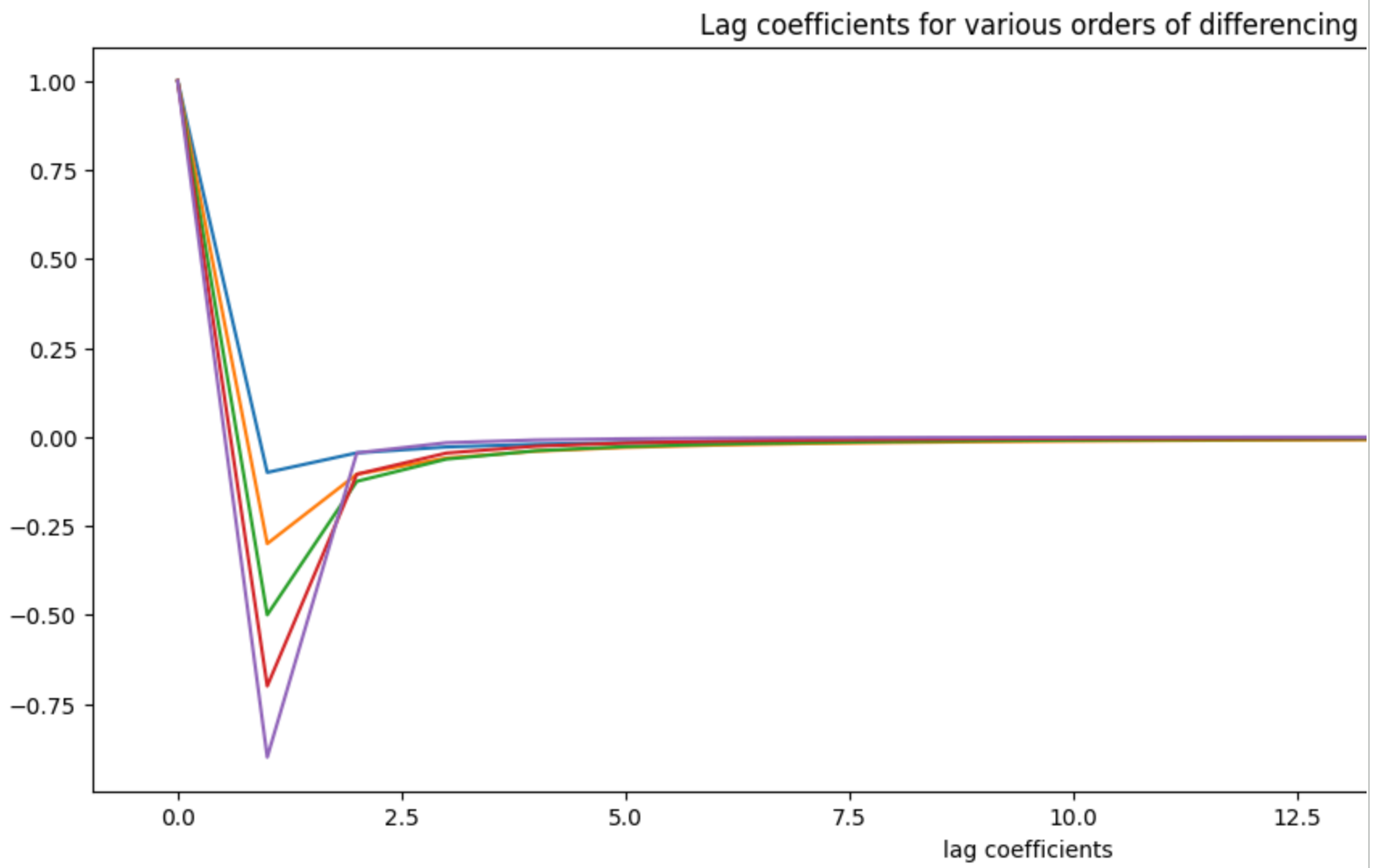
```
Returns ADF Statistic: -10.979058
Returns ADF p-value: 0.000000
```

```
Log-Returns ADF Statistic: -12.174476
Log-Returns ADF p-value: 0.000000
```

Fractional Differencing

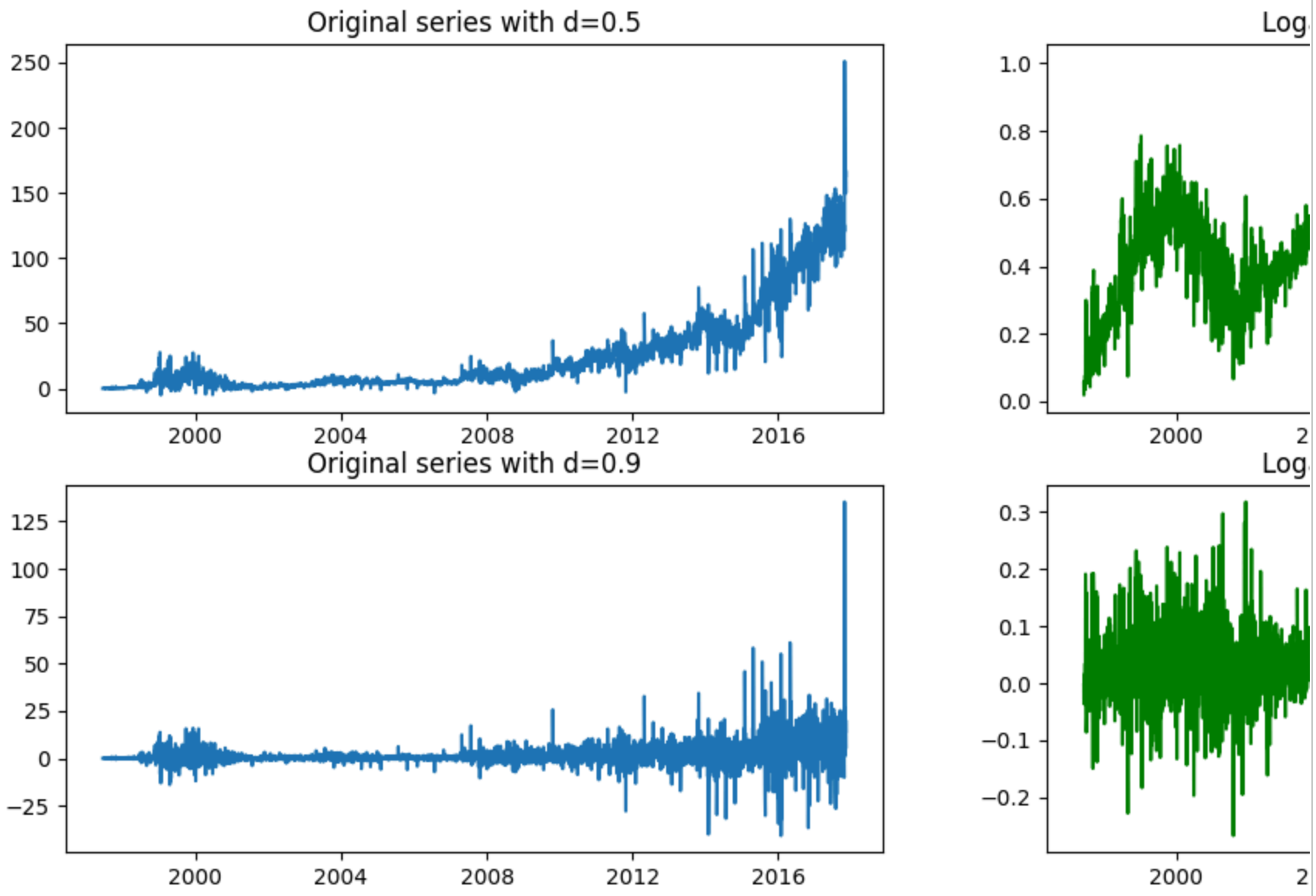
```
def getWeights(d,lags):
    # return the weights from the series expansion of the differencing operator
    # for real orders d and up to lags coefficients
    w=[1]
    for k in range(1,lags):
        w.append(-w[-1]*((d-k+1))/k)
    w=np.array(w).reshape(-1,1)
    return w
def plotWeights(dRange, lags, numberPlots):
    weights=pd.DataFrame(np.zeros((lags, numberPlots)))
    interval=np.linspace(dRange[0],dRange[1],numberPlots)
    for i, diff_order in enumerate(interval):
        weights[i]=getWeights(diff_order,lags)
    weights.columns = [round(x,2) for x in interval]
    fig=weights.plot(figsize=(15,6))
    plt.legend(title='Order of differencing')
    plt.title('Lag coefficients for various orders of differencing')
    plt.xlabel('lag coefficients')
    #plt.grid(False)
    plt.show()
def ts_differencing(series, order, lag_cutoff):
    # return the time series resulting from (fractional) differencing
    # for real orders order up to lag_cutoff coefficients

    weights=getWeights(order, lag_cutoff)
    res=0
    for k in range(lag_cutoff):
        res += weights[k]*series.shift(k).fillna(0)
    return res[lag_cutoff:]
```

```
plotWeights([0.1,0.9],20,5)
```




```
differences=[0.5,0.9]
fig, axs = plt.subplots(len(differences),2,figsize=(15,6))
for i in range(0,len(differences)):
    axs[i,0].plot(ts_differencing(df['Close'],differences[i],20))
    axs[i,0].set_title('Original series with d='+str(differences[i]))
    axs[i,1].plot(ts_differencing(df_log['Close'],differences[i],20),'g-')
    axs[i,1].set_title('Logarithmic series with d='+str(differences[i]))
```

```
plt.subplots_adjust(bottom=0.01) #increasing space between plots for aestethics
```




```
def cutoff_find(order,cutoff,start_lags): #order is our dearest d, cutoff is 1e-5 for us, and start lags is an
    val=np.inf
    lags=start_lags
```

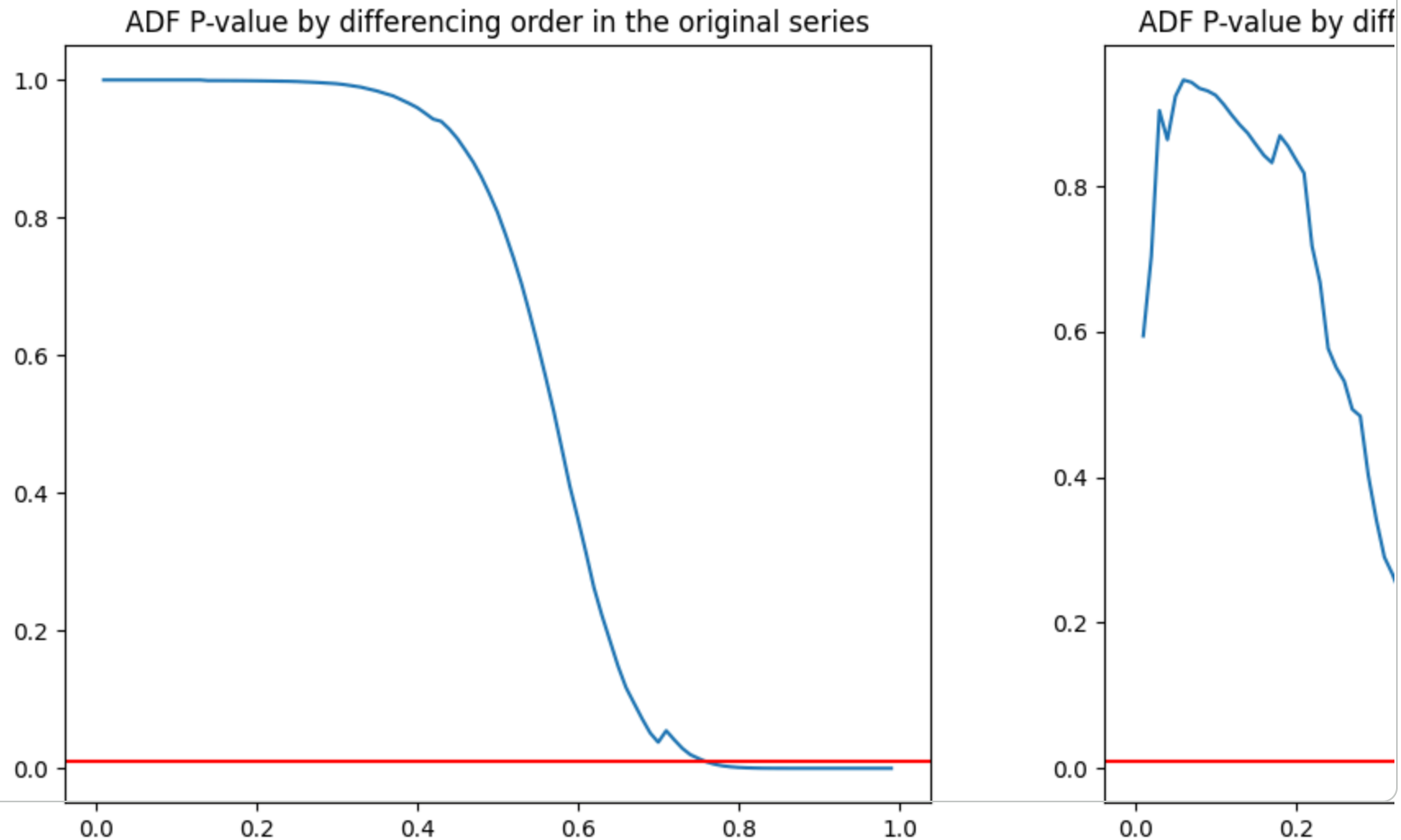
```
    while abs(val)>cutoff:
        w=getWeights(order, lags)
        val=w[len(w)-1]
        lags+=1
    return lags
```

```
def ts_differencing_tau(series, order, tau):
    # return the time series resulting from (fractional) differencing
    lag_cutoff=(cutoff_find(order,tau,1)) #finding lag cutoff with tau
    weights=getWeights(order, lag_cutoff)
    res=0
    for k in range(lag_cutoff):
        res += weights[k]*series.shift(k).fillna(0)
    return res[lag_cutoff:]
```

```
#this part takes about 20 minutes to compute
possible_d=np.divide(range(1,100),100)
tau=1e-4
original_adf_stat_holder=[None]*len(possible_d)
log_adf_stat_holder=[None]*len(possible_d)

for i in range(len(possible_d)):
    original_adf_stat_holder[i]=adfuller(ts_differencing_tau(df['Close'],possible_d[i],tau))[1]
    log_adf_stat_holder[i]=adfuller(ts_differencing_tau(df_log['Close'],possible_d[i],tau))[1]
```

```
#now the plots of the ADF p-values
fig, axs = plt.subplots(1,2,figsize=(15,6))
axs[0].plot(possible_d,original_adf_stat_holder)
axs[0].axhline(y=0.01,color='r')
axs[0].set_title('ADF P-value by differencing order in the original series')
axs[1].plot(possible_d,log_adf_stat_holder)
axs[1].axhline(y=0.01,color='r')
axs[1].set_title('ADF P-value by differencing order in the logarithmic series')
```

```
Text(0.5, 1.0, 'ADF P-value by differencing order in the logarithmic series')
```



Step 2(a)

```
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense

# ------------------------------
# 1. Download time series data
# ------------------------------
TICKER = "AMZN"
START = "2020-01-01"
END = "2025-06-30"

data = yf.download(TICKER, start=START, end=END)
prices = data["Close"].dropna()

# Convert to numpy
prices = prices.values.reshape(-1, 1)

# ------------------------------
# 2. Normalize data
# ------------------------------
scaler = MinMaxScaler()
scaled_prices = scaler.fit_transform(prices)

# ------------------------------
# 3. Build supervised dataset
# past N days → next day
# ------------------------------
N_LAGS = 10  # number of past days used as input

def create_dataset(series, n_lags):
    X, y = [], []
    for i in range(n_lags, len(series)):
        X.append(series[i-n_lags:i, 0])
        y.append(series[i, 0])
    return np.array(X), np.array(y)

X, y = create_dataset(scaled_prices, N_LAGS)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
```

```
)

# ------------------------------
# 4. Build MLP model
# ------------------------------
model = Sequential()
model.add(Dense(64, activation="relu", input_shape=(N_LAGS,)))
model.add(Dense(32, activation="relu"))
model.add(Dense(1))  # predict level

model.compile(optimizer="adam", loss="mse")

model.summary()

# ------------------------------
# 5. Train model
# ------------------------------
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_split=0.1,
    verbose=1
)

# ------------------------------
# 6. Make predictions
# ------------------------------
y_pred = model.predict(X_test)

# Invert scaling
y_test_real = scaler.inverse_transform(y_test.reshape(-1,1))
y_pred_real = scaler.inverse_transform(y_pred)

# ------------------------------
# 7. Plot prediction vs actual
# ------------------------------
plt.figure(figsize=(10,5))
plt.plot(y_test_real, label="Real Price")
```

```
plt.plot(y_pred_real, label="Predicted Price")
plt.title("MLP Prediction of AMZN Price Level")
plt.legend()
plt.show()
```

```
/tmp/ipython-input-3628711203.py:17: FutureWarning: YF.download() has changed argument auto_adjust default to T
  data = yf.download(TICKER, start=START, end=END)
[*********************100%***********************]  1 of 1 completed
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_s
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

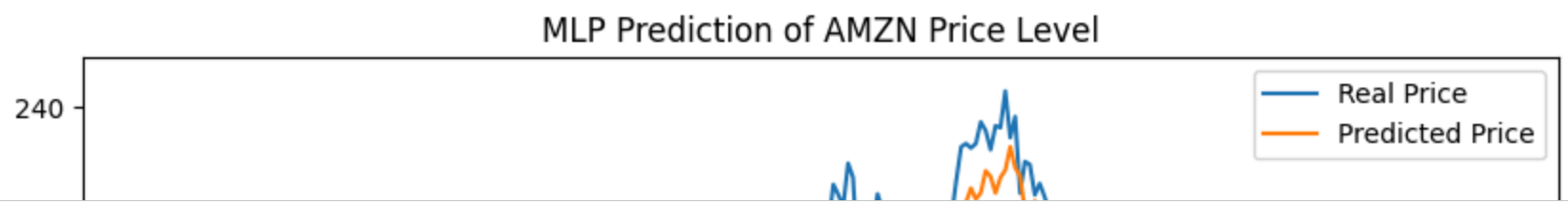
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	704
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

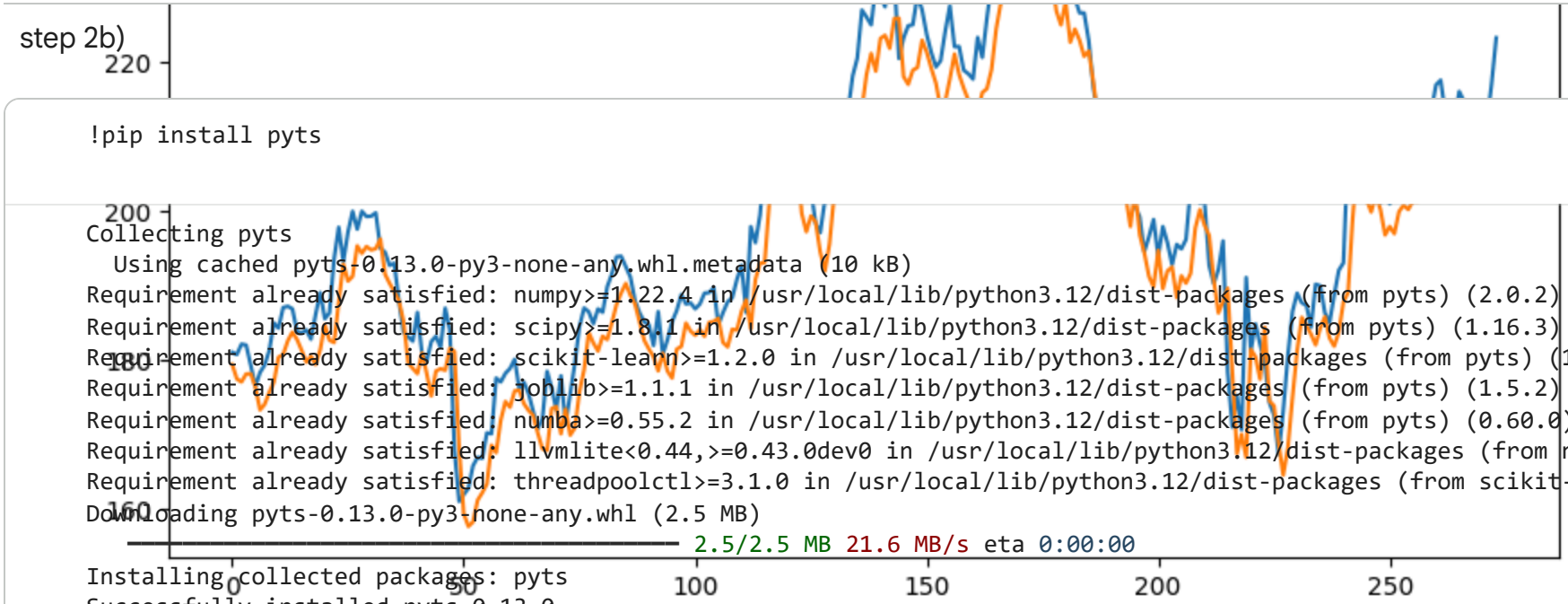
Total params: 2,817 (11.00 KB)
Trainable params: 2,817 (11.00 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0997 - val_loss: 0.0013
Epoch 2/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0011 - val_loss: 4.8629e-04
Epoch 3/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.6654e-04 - val_loss: 5.2187e-04
Epoch 4/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 6.0060e-04 - val_loss: 3.8268e-04
Epoch 5/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 6.0057e-04 - val_loss: 3.7338e-04
Epoch 6/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.3151e-04 - val_loss: 4.8492e-04
Epoch 7/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.8164e-04 - val_loss: 4.4446e-04
Epoch 8/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.4942e-04 - val_loss: 4.3335e-04
Epoch 9/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.2897e-04 - val_loss: 3.4741e-04
Epoch 10/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.6186e-04 - val_loss: 3.7370e-04
Epoch 11/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.5690e-04 - val_loss: 3.6329e-04
Epoch 12/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.0886e-04 - val_loss: 3.1863e-04
Epoch 13/30
```

```
Epoch 13/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.6274e-04 - val_loss: 5.6765e-04
Epoch 14/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.0550e-04 - val_loss: 6.0589e-04
Epoch 15/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 5.0088e-04 - val_loss: 3.1429e-04
Epoch 16/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.4744e-04 - val_loss: 3.1470e-04
Epoch 17/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.4628e-04 - val_loss: 4.1266e-04
Epoch 18/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.6180e-04 - val_loss: 3.0916e-04
Epoch 19/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.6174e-04 - val_loss: 4.9934e-04
Epoch 20/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.8449e-04 - val_loss: 3.6777e-04
Epoch 21/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.4834e-04 - val_loss: 3.1755e-04
Epoch 22/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.3315e-04 - val_loss: 9.3304e-04
Epoch 23/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.4630e-04 - val_loss: 4.5258e-04
Epoch 24/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.4602e-04 - val_loss: 3.0227e-04
Epoch 25/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.2714e-04 - val_loss: 3.1338e-04
Epoch 26/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.0252e-04 - val_loss: 4.0321e-04
Epoch 27/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.2804e-04 - val_loss: 5.8438e-04
Epoch 28/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.8987e-04 - val_loss: 3.0561e-04
Epoch 29/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.0864e-04 - val_loss: 3.1996e-04
Epoch 30/30
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.7005e-04 - val_loss: 7.9292e-04
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
```



step 2b)




```
!pip install pyts
```

```
Collecting pyts
  Using cached pyts-0.13.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: numpy>=1.22.4 in /usr/local/lib/python3.12/dist-packages (from pyts) (2.0.2)
Requirement already satisfied: scipy>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from pyts) (1.16.3)
Requirement already satisfied: scikit-learn>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from pyts) (1.6.
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from pyts) (1.5.2)
Requirement already satisfied: numba>=0.55.2 in /usr/local/lib/python3.12/dist-packages (from pyts) (0.60.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numb
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lea
Downloading pyts-0.13.0-py3-none-any.whl (2.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.5/2.5 MB 21.6 MB/s eta 0:00:00
Installing collected packages: pyts
Successfully installed pyts-0.13.0
```

```
import numpy as np
import pandas as pd

def get_fracdiff_weights(d, size):
    """Generate fractional differencing weights."""
    w = [1.0]
    for k in range(1, size):
        w.append(-w[-1] * (d - k + 1) / k)
    return np.array(w)

def fracdiff(series, d, thresh=1e-5):
    """
    Fractionally difference a time series.
    Works even when the 'fracdiff' package cannot be installed.
    """
    series = pd.Series(series)
    T = len(series)
    weights = get_fracdiff_weights(d, T)
```

```
    # Remove tiny weights to reduce computation
    weights = weights[np.abs(weights) > thresh]

    fd = np.zeros(T)
    for t in range(len(weights), T):
        fd[t] = np.sum(weights * series.values[t-len(weights)+1:t+1])
    return pd.Series(fd, index=series.index)
```

```
# ============================================================
# TASK 2B — MLP USING STATIONARY SERIES
# ============================================================

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

# ------------------------------------------------------------
# 1. LOAD BTC-USD AND CREATE STATIONARY SERIES (LOG RETURNS)
# ------------------------------------------------------------
df = yf.download("BTC-USD", period="max", progress=False)
df = df[["Close"]].dropna().tail(2000)
df.rename(columns={"Close": "price"}, inplace=True)

df["log_price"] = np.log(df["price"])
df["r1"] = df["log_price"].diff()
stationary = df["r1"].dropna()

# ADF Test
adf = adfuller(stationary)
print("ADF Statistic:", adf[0])
print("p-value:", adf[1])
```

```
# ------------------------------------------------------------
# 2. CREATE LAG FEATURES
# ------------------------------------------------------------
def create_lags(series, lags=20):
    df_lag = pd.DataFrame()
    for i in range(1, lags + 1):
        df_lag[f"lag_{i}"] = series.shift(i)
    df_lag["y"] = series
    df_lag = df_lag.dropna()
    return df_lag.drop(columns=["y"]).values, df_lag["y"].values

X, y = create_lags(stationary, 20)

# Split: 70/15/15
train_size = int(0.70 * len(X))
val_size   = int(0.15 * len(X))

X_train, y_train = X[:train_size], y[:train_size]
X_val, y_val     = X[train_size:train_size+val_size], y[train_size:train_size+val_size]
X_test, y_test   = X[train_size+val_size:], y[train_size+val_size:]

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val   = scaler.transform(X_val)
X_test  = scaler.transform(X_test)

# ------------------------------------------------------------
# 3. BUILD MLP MODEL
# ------------------------------------------------------------
def build_mlp(input_dim):
    model = models.Sequential([
        layers.Dense(128, activation="relu", input_shape=(input_dim,)),
        layers.Dropout(0.2),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(1)
    ])
```

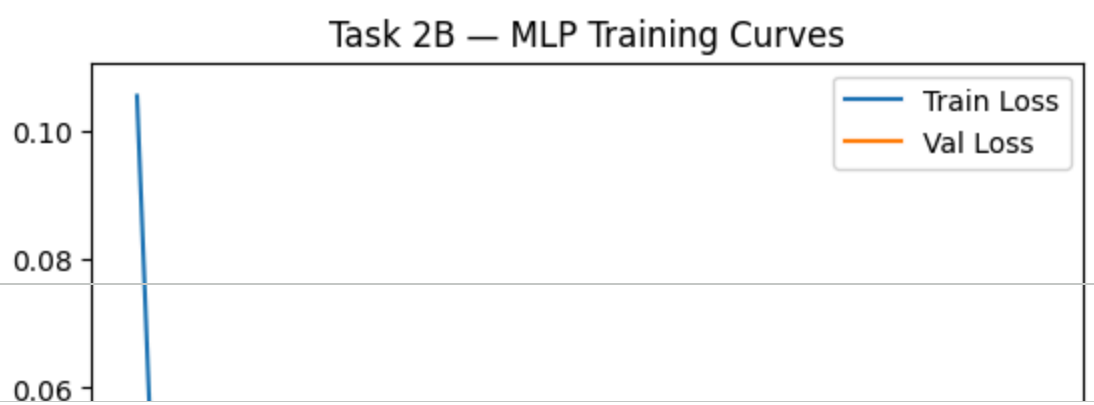
```
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
                  loss="mse", metrics=["mae"])
    return model

mlp = build_mlp(X_train.shape[1])

early_stop = callbacks.EarlyStopping(
    patience=10, restore_best_weights=True
)

# ------------------------------------------------------------
# 4. TRAIN MODEL
# ------------------------------------------------------------
history = mlp.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=200,
    batch_size=32,
    callbacks=[early_stop],
    verbose=2
)

# Training plot
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Val Loss")
plt.title("Task 2B — MLP Training Curves")
plt.legend()
plt.show()

# ------------------------------------------------------------
# 5. EVALUATE MODEL
# ------------------------------------------------------------
test_mse, test_mae = mlp.evaluate(X_test, y_test, verbose=0)
print("Task 2B — Test MSE:", test_mse)
print("Task 2B — Test MAE:", test_mae)

# Predictions plot
y_pred = mlp.predict(X_test).flatten()
```

```
plt.figure(figsize=(10,4))
plt.plot(y_test, label="Actual")
plt.plot(y_pred, label="Predicted")
plt.title("Task 2B — Predictions vs Actual")
plt.legend()
plt.show()
```

```
/tmp/ipython-input-1270011884.py:17: FutureWarning: YF.download() has changed argument auto_adjust default to T
  df = yf.download("BTC-USD", period="max", progress=False)
ADF Statistic: -46.07291460316586
p-value: 0.0
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_s
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/200
44/44 - 5s - 108ms/step - loss: 0.1055 - mae: 0.2319 - val_loss: 0.0117 - val_mae: 0.0807
Epoch 2/200
44/44 - 1s - 17ms/step - loss: 0.0401 - mae: 0.1467 - val_loss: 0.0059 - val_mae: 0.0590
Epoch 3/200
44/44 - 1s - 13ms/step - loss: 0.0243 - mae: 0.1141 - val_loss: 0.0040 - val_mae: 0.0472
Epoch 4/200
44/44 - 1s - 14ms/step - loss: 0.0149 - mae: 0.0892 - val_loss: 0.0027 - val_mae: 0.0402
Epoch 5/200
44/44 - 1s - 13ms/step - loss: 0.0103 - mae: 0.0748 - val_loss: 0.0020 - val_mae: 0.0341
Epoch 6/200
44/44 - 0s - 8ms/step - loss: 0.0069 - mae: 0.0609 - val_loss: 0.0015 - val_mae: 0.0299
Epoch 7/200
44/44 - 1s - 13ms/step - loss: 0.0057 - mae: 0.0536 - val_loss: 0.0013 - val_mae: 0.0277
Epoch 8/200
44/44 - 1s - 17ms/step - loss: 0.0052 - mae: 0.0491 - val_loss: 0.0011 - val_mae: 0.0249
Epoch 9/200
44/44 - 1s - 13ms/step - loss: 0.0033 - mae: 0.0423 - val_loss: 9.4556e-04 - val_mae: 0.0232
Epoch 10/200
44/44 - 1s - 13ms/step - loss: 0.0033 - mae: 0.0414 - val_loss: 8.6533e-04 - val_mae: 0.0220
Epoch 11/200
44/44 - 1s - 16ms/step - loss: 0.0028 - mae: 0.0378 - val_loss: 8.3831e-04 - val_mae: 0.0218
Epoch 12/200
44/44 - 1s - 31ms/step - loss: 0.0023 - mae: 0.0342 - val_loss: 7.9605e-04 - val_mae: 0.0210
Epoch 13/200
44/44 - 1s - 14ms/step - loss: 0.0022 - mae: 0.0334 - val_loss: 7.7581e-04 - val_mae: 0.0208
Epoch 14/200
44/44 - 1s - 12ms/step - loss: 0.0020 - mae: 0.0320 - val_loss: 7.4952e-04 - val_mae: 0.0202
Epoch 15/200
44/44 - 1s - 14ms/step - loss: 0.0017 - mae: 0.0300 - val_loss: 7.4617e-04 - val_mae: 0.0204
Epoch 16/200
44/44 - 1s - 13ms/step - loss: 0.0017 - mae: 0.0295 - val_loss: 7.3340e-04 - val_mae: 0.0201
Epoch 17/200
44/44 - 1s - 14ms/step - loss: 0.0016 - mae: 0.0280 - val_loss: 7.2716e-04 - val_mae: 0.0201
Epoch 18/200
44/44 - 0s - 8ms/step - loss: 0.0015 - mae: 0.0278 - val_loss: 7.1363e-04 - val_mae: 0.0197
```

```
Epoch 19/200
44/44 - 0s - 8ms/step - loss: 0.0015 - mae: 0.0268 - val_loss: 7.1216e-04 - val_mae: 0.0197
Epoch 20/200
44/44 - 1s - 14ms/step - loss: 0.0015 - mae: 0.0275 - val_loss: 7.1004e-04 - val_mae: 0.0198
Epoch 21/200
44/44 - 1s - 18ms/step - loss: 0.0013 - mae: 0.0259 - val_loss: 7.0999e-04 - val_mae: 0.0199
Epoch 22/200
44/44 - 0s - 11ms/step - loss: 0.0014 - mae: 0.0259 - val_loss: 6.9239e-04 - val_mae: 0.0194
Epoch 23/200
44/44 - 1s - 14ms/step - loss: 0.0013 - mae: 0.0252 - val_loss: 6.8682e-04 - val_mae: 0.0192
Epoch 24/200
44/44 - 0s - 11ms/step - loss: 0.0012 - mae: 0.0249 - val_loss: 6.8593e-04 - val_mae: 0.0194
Epoch 25/200
44/44 - 0s - 10ms/step - loss: 0.0012 - mae: 0.0247 - val_loss: 6.7696e-04 - val_mae: 0.0191
Epoch 26/200
44/44 - 0s - 8ms/step - loss: 0.0012 - mae: 0.0245 - val_loss: 6.7651e-04 - val_mae: 0.0192
Epoch 27/200
44/44 - 0s - 8ms/step - loss: 0.0012 - mae: 0.0245 - val_loss: 6.7626e-04 - val_mae: 0.0193
Epoch 28/200
44/44 - 0s - 9ms/step - loss: 0.0012 - mae: 0.0241 - val_loss: 6.7295e-04 - val_mae: 0.0192
Epoch 29/200
44/44 - 0s - 5ms/step - loss: 0.0012 - mae: 0.0239 - val_loss: 6.6791e-04 - val_mae: 0.0190
Epoch 30/200
44/44 - 0s - 4ms/step - loss: 0.0013 - mae: 0.0250 - val_loss: 6.6889e-04 - val_mae: 0.0191
Epoch 31/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0236 - val_loss: 6.6401e-04 - val_mae: 0.0189
Epoch 32/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0233 - val_loss: 6.6576e-04 - val_mae: 0.0190
Epoch 33/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0231 - val_loss: 6.6780e-04 - val_mae: 0.0192
Epoch 34/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0237 - val_loss: 6.6700e-04 - val_mae: 0.0190
Epoch 35/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0238 - val_loss: 6.6862e-04 - val_mae: 0.0192
Epoch 36/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0232 - val_loss: 6.6546e-04 - val_mae: 0.0190
Epoch 37/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0229 - val_loss: 6.6244e-04 - val_mae: 0.0189
Epoch 38/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0233 - val_loss: 6.6583e-04 - val_mae: 0.0191
Epoch 39/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0234 - val_loss: 6.6222e-04 - val_mae: 0.0189
Epoch 40/200
```

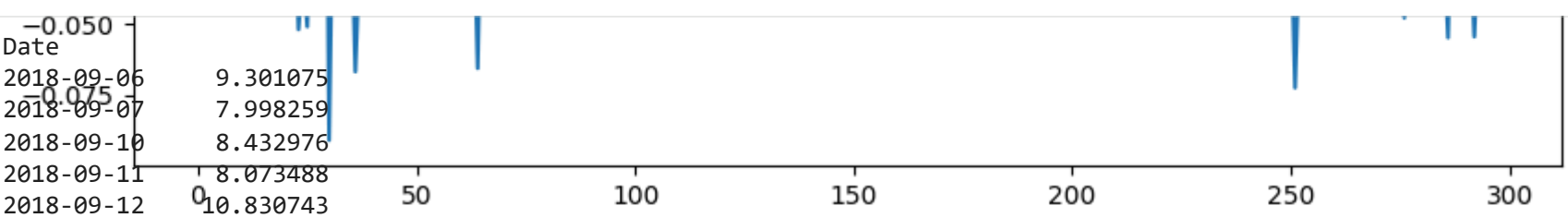
```
44/44 - 0s - 6ms/step - loss: 0.0011 - mae: 0.0230 - val_loss: 6.6235e-04 - val_mae: 0.0189
Epoch 41/200
44/44 - 0s - 6ms/step - loss: 0.0011 - mae: 0.0232 - val_loss: 6.6461e-04 - val_mae: 0.0191
Epoch 42/200
44/44 - 0s - 6ms/step - loss: 0.0010 - mae: 0.0224 - val_loss: 6.6514e-04 - val_mae: 0.0191
Epoch 43/200
44/44 - 0s - 6ms/step - loss: 0.0010 - mae: 0.0226 - val_loss: 6.6323e-04 - val_mae: 0.0190
Epoch 44/200
44/44 - 0s - 7ms/step - loss: 0.0011 - mae: 0.0229 - val_loss: 6.6078e-04 - val_mae: 0.0188
Epoch 45/200
44/44 - 0s - 6ms/step - loss: 0.0011 - mae: 0.0230 - val_loss: 6.5799e-04 - val_mae: 0.0188
Epoch 46/200
44/44 - 0s - 6ms/step - loss: 0.0011 - mae: 0.0230 - val_loss: 6.5903e-04 - val_mae: 0.0188
Epoch 47/200
44/44 - 0s - 8ms/step - loss: 0.0011 - mae: 0.0232 - val_loss: 6.7040e-04 - val_mae: 0.0193
Epoch 48/200
44/44 - 0s - 6ms/step - loss: 0.0010 - mae: 0.0227 - val_loss: 6.6312e-04 - val_mae: 0.0190
Epoch 49/200
44/44 - 0s - 4ms/step - loss: 0.0010 - mae: 0.0224 - val_loss: 6.6127e-04 - val_mae: 0.0190
Epoch 50/200
44/44 - 0s - 4ms/step - loss: 0.0011 - mae: 0.0231 - val_loss: 6.6401e-04 - val_mae: 0.0191
Epoch 51/200
44/44 - 0s - 4ms/step - loss: 0.0010 - mae: 0.0225 - val_loss: 6.6425e-04 - val_mae: 0.0191
Epoch 52/200
44/44 - 0s - 7ms/step - loss: 9.8661e-04 - mae: 0.0222 - val_loss: 6.6393e-04 - val_mae: 0.0191
Epoch 53/200
44/44 - 0s - 4ms/step - loss: 0.0010 - mae: 0.0222 - val_loss: 6.6026e-04 - val_mae: 0.0189
Epoch 54/200
44/44 - 0s - 4ms/step - loss: 9.7922e-04 - mae: 0.0223 - val_loss: 6.6193e-04 - val_mae: 0.0190
Epoch 55/200
44/44 - 0s - 4ms/step - loss: 9.9832e-04 - mae: 0.0222 - val_loss: 6.6451e-04 - val_mae: 0.0192
```



Step 2(c)

```
import pandas as pd
import numpy as np

def fracdiff(series, d, thresh=1e-4):
    w = [1.]
    k = 1
    while True:
        w_k = -w[-1] * (d - k + 1) / k
        if abs(w_k) < thresh:
            break
        w.append(w_k)
        k += 1

    w = np.array(w[::-1])
    out = pd.Series(index=series.index, dtype=float)

    for i in range(len(w), len(series)):
        out.iloc[i] = np.dot(w, series.iloc[i-len(w):i])

    return out.dropna()
```

```
fd_series = fracdiff(price, d=0.40)
print(fd_series.head())
print(fd_series.describe())
```




```
Date
2018-09-06     9.301075
2018-09-07     7.998259
2018-09-10     8.432976
2018-09-11     8.073488
2018-09-12    10.830743
dtype: float64
count    976.000000
mean       9.564283
std        5.259571
```

```
min      -12.646059
25%        6.195872
50%        9.044640
75%       12.926076
max       26.067863
dtype: float64
```

```
# Requirements: numpy, pandas, sklearn, tensorflow, matplotlib, statsmodels
import numpy as np
import pandas as pd
import math
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, accuracy_score, roc_auc_score, precision_
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
import matplotlib.pyplot as plt

# ---------------------------
# SETTINGS (change as needed)
# ---------------------------
LAGS = 20                       # number of lags (past information)
MAX_OBS = 2000                  # keep series <= 2000
TEST_SIZE = 0.15
VAL_SIZE = 0.15
SEED = 42
tf.random.set_seed(SEED); np.random.seed(SEED)

# ---------------------------
# INPUT: fd_series (pandas Series) from Step 1
# ---------------------------
# Example: fd_series = your fractionally differenced series (pandas.Series)
series = fd_series.dropna().iloc[-MAX_OBS:].copy()
series.name = 'fd_close'

# ---------------------------
# 1) Create lagged feature DataFrame
# ---------------------------
def make_lagged_df(series, lags, add_features=True):
```

```
    cols = []
    df = pd.DataFrame(index=series.index)
    for i in range(1, lags+1):
        df[f'lag_{i}'] = series.shift(i)
        cols.append(f'lag_{i}')
    df['target'] = series
    if add_features:
        # rolling features (useful)
        df['roll_mean_5'] = series.shift(1).rolling(window=5, min_periods=1).mean()
        df['roll_std_5']  = series.shift(1).rolling(window=5, min_periods=1).std().fillna(0)
        df['momentum_5']  = series.shift(1) - series.shift(6)
        cols += ['roll_mean_5','roll_std_5','momentum_5']
    df = df.dropna()
    return df, cols

df_sup, feature_cols = make_lagged_df(series, LAGS, add_features=True)
print("Supervised shape:", df_sup.shape)

# ---------------------------
# 2) Create classification target (optional)
# ---------------------------
# Binary direction:
df_sup['target_bin'] = (df_sup['target'] > 0).astype(int)

# Multi-class example using threshold (returns small -> neutral)
thr = df_sup['target'].abs().quantile(0.25)  # tuneable
def multiclass_label(x, thr):
    if x > thr: return 2   # up
    if x < -thr: return 0  # down
    return 1               # neutral
df_sup['target_multi'] = df_sup['target'].apply(lambda x: multiclass_label(x, thr))

# ---------------------------
# 3) Train/val/test split (time ordered)
# ---------------------------
n = len(df_sup)
n_test = int(math.ceil(TEST_SIZE * n))
n_val = int(math.ceil(VAL_SIZE * n))
n_train = n - n_val - n_test
```

```
train = df_sup.iloc[:n_train]
val   = df_sup.iloc[n_train:n_train+n_val]
test  = df_sup.iloc[n_train+n_val:]

X_train = train[feature_cols].values; y_train_reg = train['target'].values
X_val   = val[feature_cols].values;   y_val_reg   = val['target'].values
X_test  = test[feature_cols].values;  y_test_reg  = test['target'].values

y_train_bin = train['target_bin'].values
y_val_bin   = val['target_bin'].values
y_test_bin  = test['target_bin'].values

y_train_multi = train['target_multi'].values
y_val_multi   = val['target_multi'].values
y_test_multi  = test['target_multi'].values

print("Sizes -> Train:", X_train.shape, "Val:", X_val.shape, "Test:", X_test.shape)

# ---------------------------
# 4) Scaling
# ---------------------------
scaler_X = StandardScaler().fit(X_train)
scaler_y = StandardScaler().fit(y_train_reg.reshape(-1,1))

X_train_s = scaler_X.transform(X_train)
X_val_s   = scaler_X.transform(X_val)
X_test_s  = scaler_X.transform(X_test)

y_train_s = scaler_y.transform(y_train_reg.reshape(-1,1)).ravel()
y_val_s   = scaler_y.transform(y_val_reg.reshape(-1,1)).ravel()
y_test_s  = scaler_y.transform(y_test_reg.reshape(-1,1)).ravel()

# ---------------------------
# 5) Model builders
# ---------------------------
def build_mlp_reg(input_dim, hidden=[128,64], dropout=0.2, l2=1e-4):
    inp = layers.Input(shape=(input_dim,))
    x = inp
```

```
    for u in hidden:
        x = layers.Dense(u, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(l2))(x)
        x = layers.Dropout(dropout)(x)
    out = layers.Dense(1, activation='linear')(x)
    model = models.Model(inp, out)
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

def build_mlp_clf(input_dim, hidden=[128,64], dropout=0.2, l2=1e-4, n_classes=2):
    inp = layers.Input(shape=(input_dim,))
    x = inp
    for u in hidden:
        x = layers.Dense(u, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(l2))(x)
        x = layers.Dropout(dropout)(x)
    if n_classes == 2:
        out = layers.Dense(1, activation='sigmoid')(x)
        model = models.Model(inp, out)
        model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    else:
        out = layers.Dense(n_classes, activation='softmax')(x)
        model = models.Model(inp, out)
        model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model

# ---------------------------
# 6) Train regression MLP
# ---------------------------
reg_model = build_mlp_reg(X_train_s.shape[1])
es = callbacks.EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True)
reg_history = reg_model.fit(X_train_s, y_train_s, validation_data=(X_val_s, y_val_s),
                            epochs=200, batch_size=32, callbacks=[es], verbose=2)

# Inverse transform predictions
y_pred_s = reg_model.predict(X_test_s).ravel()
y_pred = scaler_y.inverse_transform(y_pred_s.reshape(-1,1)).ravel()

rmse = np.sqrt(mean_squared_error(y_test_reg, y_pred))
mae = mean_absolute_error(y_test_reg, y_pred)
print(f"Regressor Test RMSE: {rmse:.6f}, MAE: {mae:.6f}")
```

```
# ---------------------------
# 7) Train classifier (binary direction)
# ---------------------------
# handle class imbalance via class_weight
from sklearn.utils import class_weight
cw = class_weight.compute_class_weight('balanced', classes=np.unique(y_train_bin), y=y_train_bin)
class_weights = {i: cw[i] for i in range(len(cw))}

clf_bin = build_mlp_clf(X_train_s.shape[1], n_classes=2)
history_bin = clf_bin.fit(X_train_s, y_train_bin, validation_data=(X_val_s, y_val_bin),
                          epochs=200, batch_size=32, callbacks=[es], class_weight=class_weights, verbose=2)

# Evaluate binary classifier
y_prob = clf_bin.predict(X_test_s).ravel()
y_pred_bin = (y_prob > 0.5).astype(int)
acc = accuracy_score(y_test_bin, y_pred_bin)
try:
    auc = roc_auc_score(y_test_bin, y_prob)
except:
    auc = np.nan
prec, rec, f1, _ = precision_recall_fscore_support(y_test_bin, y_pred_bin, average='binary')
print(f"Binary Classifier -> Acc: {acc:.3f}, AUC: {auc:.3f}, Prec: {prec:.3f}, Rec: {rec:.3f}, F1: {f1:.3f}")

# ---------------------------
# 8) (Optional) Multi-class classifier
# ---------------------------
clf_multi = build_mlp_clf(X_train_s.shape[1], n_classes=3)
history_multi = clf_multi.fit(X_train_s, y_train_multi, validation_data=(X_val_s, y_val_multi),
                              epochs=200, batch_size=32, callbacks=[es], verbose=2)
acc_multi = clf_multi.evaluate(X_test_s, y_test_multi, verbose=0)[1]
print(f"Multi-class test accuracy: {acc_multi:.3f}")

# ---------------------------
# 9) Quick diagnostics plots
# ---------------------------
plt.figure(figsize=(12,4))
plt.plot(reg_history.history['loss'], label='train loss (reg)')
plt.plot(reg_history.history['val_loss'], label='val loss (reg)')
```

```
plt.yscale('log'); plt.legend(); plt.title('Regression training')
plt.show()

plt.figure(figsize=(12,4))
plt.plot(history_bin.history['loss'], label='train loss (bin)')
plt.plot(history_bin.history['val_loss'], label='val loss (bin)')
plt.yscale('log'); plt.legend(); plt.title('Binary classifier training')
plt.show()

# ---------------------------
# 10) Simple backtest stub (directional)
# ---------------------------
# naive strategy: go long if classifier predicts up, short if predicts down (binary)
# This is illustrative only; add transaction costs, slippage, sizing in real tests.
test_dates = test.index
signal = y_pred_bin  # from binary classifier

# compute simple returns of original price if you had original close series aligned:
# Assuming 'orig_price' is a pandas Series aligned with 'series' index and same slicing
# forward_return = orig_price.pct_change().shift(-1).loc[test_dates]  # example
# strat_ret = signal * forward_return
# cumulative = (1 + strat_ret.fillna(0)).cumprod()
# cumulative.plot(); plt.title('Strategy cumulative'); plt.show()
```

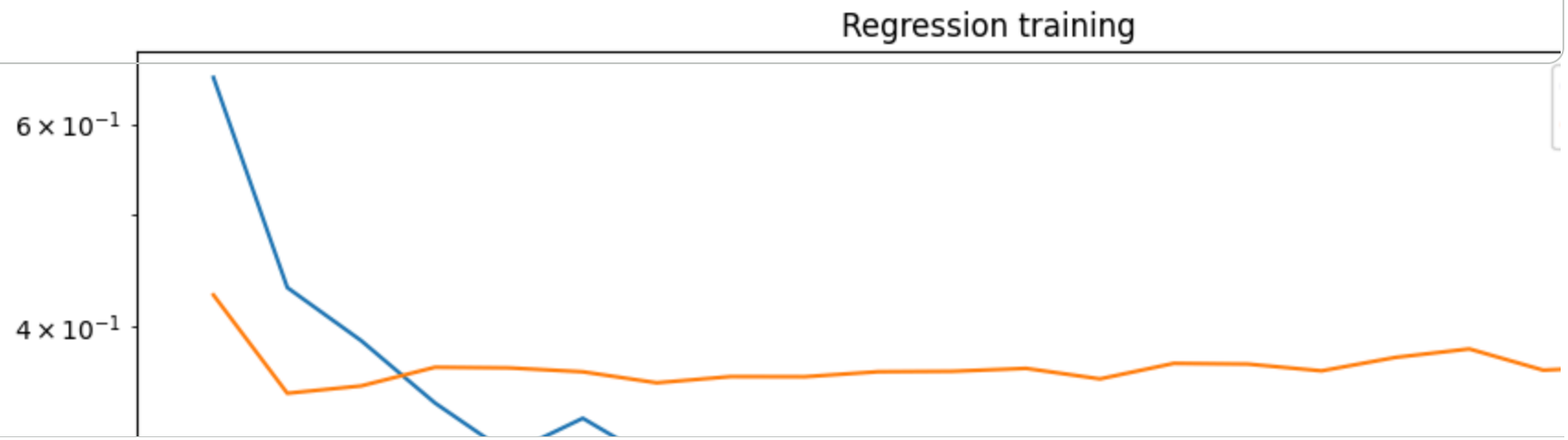
```
Supervised shape: (956, 24)
Sizes -> Train: (668, 23) Val: (144, 23) Test: (144, 23)
Epoch 1/200
21/21 - 2s - 72ms/step - loss: 0.6583 - mae: 0.6133 - val_loss: 0.4264 - val_mae: 0.4873
Epoch 2/200
21/21 - 0s - 6ms/step - loss: 0.4327 - mae: 0.4745 - val_loss: 0.3503 - val_mae: 0.4338
Epoch 3/200
21/21 - 0s - 6ms/step - loss: 0.3892 - mae: 0.4511 - val_loss: 0.3555 - val_mae: 0.4357
Epoch 4/200
21/21 - 0s - 10ms/step - loss: 0.3436 - mae: 0.4291 - val_loss: 0.3691 - val_mae: 0.4462
Epoch 5/200
21/21 - 0s - 11ms/step - loss: 0.3105 - mae: 0.4031 - val_loss: 0.3686 - val_mae: 0.4383
Epoch 6/200
21/21 - 0s - 6ms/step - loss: 0.3333 - mae: 0.4143 - val_loss: 0.3657 - val_mae: 0.4382
Epoch 7/200
21/21 - 0s - 6ms/step - loss: 0.3056 - mae: 0.3911 - val_loss: 0.3578 - val_mae: 0.4314
Epoch 8/200
21/21 - 0s - 6ms/step - loss: 0.2921 - mae: 0.3853 - val_loss: 0.3622 - val_mae: 0.4334
Epoch 9/200
21/21 - 0s - 6ms/step - loss: 0.2826 - mae: 0.3825 - val_loss: 0.3621 - val_mae: 0.4321
Epoch 10/200
21/21 - 0s - 6ms/step - loss: 0.2899 - mae: 0.3880 - val_loss: 0.3658 - val_mae: 0.4337
Epoch 11/200
21/21 - 0s - 6ms/step - loss: 0.2746 - mae: 0.3713 - val_loss: 0.3661 - val_mae: 0.4407
Epoch 12/200
21/21 - 0s - 8ms/step - loss: 0.2619 - mae: 0.3682 - val_loss: 0.3682 - val_mae: 0.4398
Epoch 13/200
21/21 - 0s - 6ms/step - loss: 0.2698 - mae: 0.3722 - val_loss: 0.3606 - val_mae: 0.4374
Epoch 14/200
21/21 - 0s - 7ms/step - loss: 0.2767 - mae: 0.3828 - val_loss: 0.3719 - val_mae: 0.4395
Epoch 15/200
21/21 - 0s - 16ms/step - loss: 0.2740 - mae: 0.3663 - val_loss: 0.3714 - val_mae: 0.4459
Epoch 16/200
21/21 - 0s - 9ms/step - loss: 0.2605 - mae: 0.3712 - val_loss: 0.3664 - val_mae: 0.4343
Epoch 17/200
21/21 - 0s - 15ms/step - loss: 0.2583 - mae: 0.3594 - val_loss: 0.3764 - val_mae: 0.4439
Epoch 18/200
21/21 - 0s - 9ms/step - loss: 0.2529 - mae: 0.3608 - val_loss: 0.3828 - val_mae: 0.4519
Epoch 19/200
21/21 - 0s - 10ms/step - loss: 0.2460 - mae: 0.3583 - val_loss: 0.3669 - val_mae: 0.4446
Epoch 20/200
21/21 - 0s - 15ms/step - loss: 0.2395 - mae: 0.3567 - val_loss: 0.3694 - val_mae: 0.4477
```

```
Epoch 21/200
21/21 - 0s - 14ms/step - loss: 0.2361 - mae: 0.3390 - val_loss: 0.3769 - val_mae: 0.4445
Epoch 22/200
21/21 - 0s - 10ms/step - loss: 0.2361 - mae: 0.3482 - val_loss: 0.3674 - val_mae: 0.4470
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
Regressor Test RMSE: 4.789804, MAE: 3.825200
Epoch 1/200
21/21 - 2s - 78ms/step - accuracy: 0.8563 - loss: 0.6193 - val_accuracy: 1.0000 - val_loss: 0.1120
Epoch 2/200
21/21 - 0s - 6ms/step - accuracy: 0.8503 - loss: 0.4229 - val_accuracy: 1.0000 - val_loss: 0.0617
Epoch 3/200
21/21 - 0s - 6ms/step - accuracy: 0.8608 - loss: 0.3144 - val_accuracy: 1.0000 - val_loss: 0.0457
Epoch 4/200
21/21 - 0s - 6ms/step - accuracy: 0.8728 - loss: 0.2593 - val_accuracy: 1.0000 - val_loss: 0.0372
Epoch 5/200
21/21 - 0s - 6ms/step - accuracy: 0.8713 - loss: 0.2241 - val_accuracy: 1.0000 - val_loss: 0.0297
Epoch 6/200
21/21 - 0s - 6ms/step - accuracy: 0.9147 - loss: 0.1641 - val_accuracy: 1.0000 - val_loss: 0.0245
Epoch 7/200
21/21 - 0s - 6ms/step - accuracy: 0.9296 - loss: 0.1614 - val_accuracy: 1.0000 - val_loss: 0.0215
Epoch 8/200
21/21 - 0s - 7ms/step - accuracy: 0.9416 - loss: 0.1345 - val_accuracy: 1.0000 - val_loss: 0.0194
Epoch 9/200
21/21 - 0s - 6ms/step - accuracy: 0.9326 - loss: 0.1477 - val_accuracy: 1.0000 - val_loss: 0.0175
Epoch 10/200
21/21 - 0s - 6ms/step - accuracy: 0.9476 - loss: 0.0955 - val_accuracy: 1.0000 - val_loss: 0.0162
Epoch 11/200
21/21 - 0s - 6ms/step - accuracy: 0.9731 - loss: 0.0871 - val_accuracy: 1.0000 - val_loss: 0.0156
Epoch 12/200
21/21 - 0s - 14ms/step - accuracy: 0.9656 - loss: 0.1175 - val_accuracy: 1.0000 - val_loss: 0.0155
Epoch 13/200
21/21 - 1s - 33ms/step - accuracy: 0.9611 - loss: 0.0956 - val_accuracy: 1.0000 - val_loss: 0.0151
Epoch 14/200
21/21 - 1s - 27ms/step - accuracy: 0.9536 - loss: 0.0992 - val_accuracy: 1.0000 - val_loss: 0.0150
Epoch 15/200
21/21 - 0s - 20ms/step - accuracy: 0.9671 - loss: 0.0822 - val_accuracy: 1.0000 - val_loss: 0.0147
Epoch 16/200
21/21 - 0s - 12ms/step - accuracy: 0.9491 - loss: 0.1143 - val_accuracy: 1.0000 - val_loss: 0.0147
Epoch 17/200
21/21 - 0s - 19ms/step - accuracy: 0.9731 - loss: 0.0639 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 18/200
21/21 - 0s - 18ms/step - accuracy: 0.9775 - loss: 0.0799 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 19/200
```

```
21/21 - 1s - 29ms/step - accuracy: 0.9775 - loss: 0.0528 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 20/200
21/21 - 0s - 21ms/step - accuracy: 0.9835 - loss: 0.0496 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 21/200
21/21 - 1s - 32ms/step - accuracy: 0.9850 - loss: 0.0571 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 22/200
21/21 - 1s - 25ms/step - accuracy: 0.9790 - loss: 0.0533 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 23/200
21/21 - 0s - 21ms/step - accuracy: 0.9850 - loss: 0.0595 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 24/200
21/21 - 1s - 26ms/step - accuracy: 0.9775 - loss: 0.0550 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 25/200
21/21 - 0s - 11ms/step - accuracy: 0.9850 - loss: 0.0512 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 26/200
21/21 - 0s - 19ms/step - accuracy: 0.9895 - loss: 0.0361 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 27/200
21/21 - 0s - 19ms/step - accuracy: 0.9790 - loss: 0.0727 - val_accuracy: 1.0000 - val_loss: 0.0143
Epoch 28/200
21/21 - 0s - 23ms/step - accuracy: 0.9805 - loss: 0.0404 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 29/200
21/21 - 0s - 23ms/step - accuracy: 0.9910 - loss: 0.0438 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 30/200
21/21 - 1s - 28ms/step - accuracy: 0.9880 - loss: 0.0382 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 31/200
21/21 - 0s - 24ms/step - accuracy: 0.9910 - loss: 0.0384 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 32/200
21/21 - 0s - 16ms/step - accuracy: 0.9895 - loss: 0.0339 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 33/200
21/21 - 0s - 18ms/step - accuracy: 0.9910 - loss: 0.0424 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 34/200
21/21 - 1s - 27ms/step - accuracy: 0.9895 - loss: 0.0329 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 35/200
21/21 - 0s - 19ms/step - accuracy: 0.9865 - loss: 0.0524 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 36/200
21/21 - 1s - 26ms/step - accuracy: 0.9895 - loss: 0.0322 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 37/200
21/21 - 0s - 16ms/step - accuracy: 0.9940 - loss: 0.0345 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 38/200
21/21 - 0s - 19ms/step - accuracy: 0.9910 - loss: 0.0316 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 39/200
21/21 - 0s - 22ms/step - accuracy: 0.9940 - loss: 0.0299 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 40/200
21/21 - 0s - 13ms/step - accuracy: 0.9940 - loss: 0.0240 - val_accuracy: 1.0000 - val_loss: 0.0145
```

```
Epoch 41/200
21/21 - 0s - 19ms/step - accuracy: 0.9940 - loss: 0.0279 - val_accuracy: 1.0000 - val_loss: 0.0145
Epoch 42/200
21/21 - 1s - 24ms/step - accuracy: 0.9970 - loss: 0.0278 - val_accuracy: 1.0000 - val_loss: 0.0145
WARNING:tensorflow:5 out of the last 16 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one
1/5 ━━━━━━━━━━━━━━━━━━━━ 0s 208ms/stepWARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrai
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step
Binary Classifier -> Acc: 0.806, AUC: 0.794, Prec: 0.867, Rec: 0.897, F1: 0.881
Epoch 1/200
21/21 - 3s - 155ms/step - accuracy: 0.7290 - loss: 0.6505 - val_accuracy: 0.9861 - val_loss: 0.1867
Epoch 2/200
21/21 - 1s - 34ms/step - accuracy: 0.7650 - loss: 0.4565 - val_accuracy: 0.9861 - val_loss: 0.1480
Epoch 3/200
21/21 - 0s - 6ms/step - accuracy: 0.8219 - loss: 0.4071 - val_accuracy: 0.9861 - val_loss: 0.1346
Epoch 4/200
21/21 - 0s - 7ms/step - accuracy: 0.8084 - loss: 0.4113 - val_accuracy: 0.9861 - val_loss: 0.1284
Epoch 5/200
21/21 - 0s - 6ms/step - accuracy: 0.8234 - loss: 0.4043 - val_accuracy: 0.9861 - val_loss: 0.1241
Epoch 6/200
21/21 - 0s - 6ms/step - accuracy: 0.8503 - loss: 0.3643 - val_accuracy: 0.9861 - val_loss: 0.1210
Epoch 7/200
21/21 - 0s - 7ms/step - accuracy: 0.8353 - loss: 0.3659 - val_accuracy: 0.9861 - val_loss: 0.1182
Epoch 8/200
21/21 - 0s - 6ms/step - accuracy: 0.8383 - loss: 0.3578 - val_accuracy: 0.9861 - val_loss: 0.1177
Epoch 9/200
21/21 - 0s - 9ms/step - accuracy: 0.8473 - loss: 0.3419 - val_accuracy: 0.9861 - val_loss: 0.1164
Epoch 10/200
21/21 - 0s - 11ms/step - accuracy: 0.8293 - loss: 0.3531 - val_accuracy: 0.9861 - val_loss: 0.1163
Epoch 11/200
21/21 - 0s - 9ms/step - accuracy: 0.8353 - loss: 0.3394 - val_accuracy: 0.9861 - val_loss: 0.1157
Epoch 12/200
21/21 - 0s - 15ms/step - accuracy: 0.8308 - loss: 0.3486 - val_accuracy: 0.9861 - val_loss: 0.1170
Epoch 13/200
21/21 - 0s - 13ms/step - accuracy: 0.8533 - loss: 0.3273 - val_accuracy: 0.9861 - val_loss: 0.1187
Epoch 14/200
21/21 - 0s - 10ms/step - accuracy: 0.8593 - loss: 0.3216 - val_accuracy: 0.9861 - val_loss: 0.1189
Epoch 15/200
21/21 - 0s - 10ms/step - accuracy: 0.8473 - loss: 0.3296 - val_accuracy: 0.9861 - val_loss: 0.1184
Epoch 16/200
21/21 - 0s - 9ms/step - accuracy: 0.8428 - loss: 0.3165 - val_accuracy: 0.9861 - val_loss: 0.1205
Epoch 17/200
21/21 - 0s - 15ms/step - accuracy: 0.8503 - loss: 0.3162 - val_accuracy: 0.9861 - val_loss: 0.1209
```

```
Epoch 18/200
21/21 - 0s - 9ms/step - accuracy: 0.8338 - loss: 0.3175 - val_accuracy: 0.9861 - val_loss: 0.1225
Epoch 19/200
21/21 - 0s - 6ms/step - accuracy: 0.8683 - loss: 0.2999 - val_accuracy: 0.9861 - val_loss: 0.1243
Epoch 20/200
21/21 - 0s - 6ms/step - accuracy: 0.8443 - loss: 0.3049 - val_accuracy: 0.9861 - val_loss: 0.1276
Epoch 21/200
21/21 - 0s - 6ms/step - accuracy: 0.8593 - loss: 0.2964 - val_accuracy: 0.9861 - val_loss: 0.1296
Epoch 22/200
21/21 - 0s - 6ms/step - accuracy: 0.8638 - loss: 0.3017 - val_accuracy: 0.9861 - val_loss: 0.1304
Epoch 23/200
21/21 - 0s - 7ms/step - accuracy: 0.8608 - loss: 0.2916 - val_accuracy: 0.9861 - val_loss: 0.1320
Epoch 24/200
21/21 - 0s - 6ms/step - accuracy: 0.8593 - loss: 0.2907 - val_accuracy: 0.9861 - val_loss: 0.1350
Epoch 25/200
21/21 - 0s - 6ms/step - accuracy: 0.8683 - loss: 0.2898 - val_accuracy: 0.9861 - val_loss: 0.1373
Epoch 26/200
21/21 - 0s - 6ms/step - accuracy: 0.8728 - loss: 0.2846 - val_accuracy: 0.9861 - val_loss: 0.1408
Epoch 27/200
21/21 - 0s - 6ms/step - accuracy: 0.8638 - loss: 0.2844 - val_accuracy: 0.9861 - val_loss: 0.1407
Epoch 28/200
21/21 - 0s - 6ms/step - accuracy: 0.8533 - loss: 0.2826 - val_accuracy: 0.9861 - val_loss: 0.1427
Epoch 29/200
21/21 - 0s - 6ms/step - accuracy: 0.8683 - loss: 0.2833 - val_accuracy: 0.9861 - val_loss: 0.1467
Epoch 30/200
21/21 - 0s - 6ms/step - accuracy: 0.8668 - loss: 0.2900 - val_accuracy: 0.9861 - val_loss: 0.1456
Epoch 31/200
21/21 - 0s - 7ms/step - accuracy: 0.8728 - loss: 0.2742 - val_accuracy: 0.9861 - val_loss: 0.1465
Multi-class test accuracy: 0.694
```



Step 3(a)

```
import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt

# ==========================================================
# 1. LOAD TIME SERIES
# ==========================================================
TICKER = "AMZN"
START = "2020-01-01"
END = "2025-06-30"

data = yf.download(TICKER, start=START, end=END)
ts = data["Close"].dropna().values

print("Loaded series length:", len(ts))

# ==========================================================
# 2. NORMALIZE LEVELS
# ==========================================================
scaler = MinMaxScaler()
ts_scaled = scaler.fit_transform(ts.reshape(-1, 1)).flatten()

# ==========================================================
# 3. BUILD SUPERVISED DATASET
# ==========================================================
WINDOW = 30

X_raw = []
y = []

for i in range(len(ts_scaled) - WINDOW):
```

```
    X_raw.append(ts_scaled[i:i + WINDOW])
    y.append(ts_scaled[i + WINDOW])

X_raw = np.array(X_raw)
y = np.array(y)

print("Raw sliding windows:", X_raw.shape)

# ============================================================
# 4. MANUAL GAF IMPLEMENTATION (GASF)
# ============================================================
def gaf_transform(series_batch):
    """
    series_batch: shape (n_samples, WINDOW)
    Output: GAF images (GASF): (n_samples, WINDOW, WINDOW)
    """
    eps = 1e-8
    X = np.clip(series_batch, -1 + eps, 1 - eps)

    phi = np.arccos(X)
    gaf = np.cos(phi[:, :, None] + phi[:, None, :])  # broadcast sum

    return gaf

# Generate GAF images
X_gaf = gaf_transform(X_raw)
X_gaf = X_gaf.reshape(X_gaf.shape[0], WINDOW, WINDOW, 1)

print("GAF images shape:", X_gaf.shape)

# ============================================================
# 5. TRAIN TEST SPLIT
# ============================================================
split = int(0.8 * len(X_gaf))
X_train, X_test = X_gaf[:split], X_gaf[split:]
y_train, y_test = y[:split], y[split:]

# ============================================================
# 6. CNN MODEL
```